Face Verification and Geotagging

Panel Notes - How the System Works | Capstone Project

1. Face Verification - "Is this really the employee?"

Question	Simple Answer
Where does recognition happen?	On the SERVER, not the phone. The phone only takes pictures and sends them; the server does all the AI work using a library called face-api.js.
How is an employee enrolled?	HR takes one reference photo of the employee in the admin website (one face only, clear and large enough, no eyewear, consent given). The system turns that face into a list of 128 numbers (a "face fingerprint") and saves it.
What happens at clock-in?	The phone takes a selfie. The server brightens it if it is too dark, turns it into the same kind of 128 numbers, then measures how far apart the two fingerprints are.
What if the room is too dark?	The server measures how bright the face is, on a scale of 0 (black) to 255 (white). If it is less than 60, the app asks for more light before taking the photo.
How is the decision made?	Distance of 0.45 or less: APPROVED. More than 0.45 up to 0.6: FLAGGED for review (borderline). More than 0.6: FLAGGED for review (mismatch). No face found: REJECTED, retake in good lighting. The 0.45 and 0.6 limits are the standard values recommended by the library itself.
What if someone else uses my account?	The face does not match, so the scan is FLAGGED with the photo, location and score for the supervisor or admin to decide. On the 3rd flag in one day, the supervisor (or HR) and the employee are notified, and attendance is put on hold until reviewed.

2. Blink Check - "Is this a live person, not a photo?"

The idea in one sentence: a printed photo or a picture on another phone can't blink - so we require one real blink, and the photo is taken the moment it happens.

How it works, step by step:

1. While the employee looks at the camera, the phone sends small pictures (192 pixels wide) to the server, one right after another.
2. For each picture, the server measures how OPEN the eyes are - a standard measurement called the Eye Aspect Ratio (EAR): eye height divided by eye width.
3. The first open-eye picture sets how open THIS person's eyes normally are (their "normal level"). Later pictures keep it updated.
4. The moment one picture reads less than 97% of the normal level, that is a blink - liveness passed. If the very first picture already shows clearly closed eyes (EAR of 0.18 or less), it counts right away.
5. The photo is taken immediately after the blink. A printed photo can never get past step 4.

> Why our threshold looks small (only a 3% drop): textbooks say a blink makes the EAR drop by about 30%, but that assumes perfect eye tracking. On real phones, the eye-tracking model can't precisely follow a closed eyelid, so real blinks only dropped by 5 to 8% in our testing. We set the threshold using real phone data, not the textbook.

3. How the Numbers Are Computed (with examples)

A. Eye openness (Eye Aspect Ratio) - used for the blink check

Step	In words	Example
1. Measure the eye	The server finds 6 points around each eye. It measures the eye's height in two places and its width once.	Height 1 is 9 pixels, height 2 is 9 pixels, width is 30 pixels
2. Add the two heights	Height 1 plus height 2	9 plus 9 equals 18
3. Double the width	Width times 2	30 times 2 equals 60
4. Divide	Step 2 divided by step 3	18 divided by 60 equals **0.30 (eye open)**
Blink test	Is this picture less than 97% of the normal level?	Normal level 0.30. 97% of 0.30 is 0.291. A picture reading 0.28 is less than 0.291, so it is **a blink**.

B. Face brightness - used for the lighting check

Step	In words	Example (one pixel: red 100, green 80, blue 60)
1. Weigh each color	Our eyes see green as brightest, so each color counts differently: red times 0.299, green times 0.587, blue times 0.114	100 times 0.299 equals 29.9; 80 times 0.587 equals 47.0; 60 times 0.114 equals 6.8
2. Add them	The total is that pixel's brightness	29.9 plus 47.0 plus 6.8 equals 83.7
3. Average the face	Do this for every pixel of the face and take the average	Average of 83.7 is at least 60, so **bright enough**

C. Face distance - how different two faces are

Step	In words	Example (using only 3 of the 128 numbers)
1. Subtract	For each number, subtract the new photo's value from the enrolled value	Enrolled 0.10, 0.20, 0.30. New photo 0.13, 0.16, 0.30. Differences: 0.03, 0.04, 0
2. Square	Multiply each difference by itself (so negatives don't cancel out)	0.0009, 0.0016, 0
3. Add	Add all the squared differences together	0.0009 plus 0.0016 plus 0 equals 0.0025
4. Square root	Take the square root of the total	Square root of 0.0025 equals **0.05 (very alike)**
Meaning	The real check does this for all 128 numbers. The smaller the distance, the more alike the two faces are. A distance of 0 means identical.	

D. Similarity score - the percentage shown to reviewers

Step	In words	Example (distance 0.30)
1. Divide	Distance divided by 1.2	0.30 divided by 1.2 equals 0.25
2. Subtract from 1	1 minus the result of step 1	1 minus 0.25 equals 0.75
3. Make it a percent	Times 100	0.75 times 100 equals **75% similar**
Decision	Distance 0.30 is 0.45 or less, so the scan is **APPROVED**. Using the same steps: distance 0.45 gives about 63%, and distance 0.6 gives 50%. So 63% or more is approved, and 50 to 62% is flagged as borderline.	

E. Distance from the work site (Haversine formula) - used for geotagging

The Earth is round, so the server uses the Haversine formula, the standard way to measure the distance between two GPS points along the Earth's surface. An easy way to picture it: how far north or south the employee is, and how far east or west, form the two sides of a right triangle. The distance is the long side.

Step	In words	Example (Agoo plant pin 16.358048, 120.353511; employee at 16.362048, 120.355511)
1. North-south gap	Difference in latitude. One degree of latitude is about 111,195 meters.	0.004 degrees times 111,195 equals about 445 meters
2. East-west gap	Difference in longitude. Near Agoo, one degree of longitude is a bit shorter (about 96% of 111,195 meters) because the Earth narrows away from the equator.	0.002 degrees times 106,700 equals about 213 meters
3. Combine	Square both sides, add them, then take the square root	445 times 445 equals 198,025; 213 times 213 equals 45,369; together 243,394; square root equals **about 493 meters**
Exact formula	The server uses the full Haversine formula with sine, cosine, and the Earth's radius of 6,371,000 meters. For distances this short it gives the same answer: 493 meters.	

4. Geotagging - "Is the employee actually at work?"

Question	Simple Answer
How is a work area defined?	The admin drops a pin on a map and sets a CIRCLE RADIUS around it. That circle is the geofence. Each area is Office (one record per day) or Field (several site visits per day).
How do we know they're inside?	The phone sends its GPS coordinates. The server computes the real-world distance between the employee and the pin using the HAVERSINE FORMULA (see 3E above).
Two conditions must pass	(1) The phone's GPS accuracy must be good enough - an error of 50 meters or less by default. (2) The distance from the pin must be equal to or less than the radius. If GPS is unreliable, the system rejects rather than guesses. The face is only checked after both pass.
Checked on the phone or the server?	BOTH - the phone checks first for instant feedback ("outside" or "weak signal"), but the server re-checks every submission, so a tampered app cannot bypass it.
Which area is checked?	Every employee belongs to the Agoo default area and can have one other area. If they have two, the server checks the one they are nearest to.
What about the web portal?	A laptop has no GPS chip, so on the web the allowed GPS error is raised to 2,000 meters. The distance rule stays the same.

Examples (Agoo plant, radius 1,000 meters, allowed GPS error 50 meters):

Scan	Result
GPS error 12 m, distance 493 m	12 is less than 50, so GPS is good. 493 is less than 1,000, so the employee is inside. **ACCEPTED**
GPS error 10 m, distance 1,334 m	GPS is good, but 1,334 is more than 1,000, so the employee is outside. **REJECTED (outside)**
GPS error 80 m, distance 493 m	80 is more than 50, so the GPS reading can't be trusted. **REJECTED (weak GPS)**, even though the distance would pass

About the radius:

- Allowed range in the form: 25 to 1,000 meters (default 120 meters). The Agoo plant uses 1,000 meters.
- Recommended: 50 to 150 meters for an office building.
- Why not smaller than 25 meters? Phone GPS is usually only accurate to about 5 to 50 meters. A circle smaller than the GPS error means real employees get rejected while standing at the front door.
- Why not 1,000 meters for an office? At that size, anyone in the whole neighborhood would pass - it only makes sense for large sites.

5. The Full Clock-In Flow (what the demo shows)

1. Employee opens the app and starts clock-in. The GPS reading starts in the background right away.
2. BLINK CHECK - proves a live person is present.
3. PHOTO TAKEN the moment the blink is seen, shown with the time, date, address and a mini-map.
4. Server checks that the LOCATION IS INSIDE the assigned geofence.
5. Server checks that the FACE MATCHES the enrolled profile.
6. Both pass: attendance recorded. Face mismatch inside the area: flagged for a person to review. Wrong location or weak GPS: rejected outright.

6. What Changed Since July - Faster Face Detection

The match limits (0.45 and 0.6) and the one-blink rule are the same. What changed is how fast the scan reaches a result.

Change	Before (July)	Now	Why it is faster
Model for blink pictures	Full eye-tracking model, picture checked at 416 pixels	Small, fast eye-tracking model, picture checked at 192 pixels	Less server work per picture, so more pictures per second and fewer missed blinks
Picture sent to server	480 pixels wide, 70% quality	192 pixels wide, 45% quality	About 80% fewer pixels to compress, upload and read
Camera size while scanning	Full camera resolution	Smallest size that still gives a 1,080-pixel photo	Each picture is taken and compressed faster on the phone
Wait between face-tracking requests	250 milliseconds	20 milliseconds	The face box turns green almost as soon as the server replies
Pictures needed before a blink counts	2 open-eye pictures	1 picture; a clearly closed first picture counts right away	A blink can be caught from the very first picture
Blink threshold	Less than 95% of normal	Less than 97% of normal	Real blinks (5 to 8% drop) are caught on the first try
After the blink	0.8-second "hold still" countdown, then the photo	Photo taken the instant the blink is seen	Removes the 0.8-second wait
GPS reading	Started after the photo; waited several seconds	Started when the camera opens	GPS is usually ready by the time the blink is done
Photo stamp	Address lookup and stamp drawn into the photo before sending	Original photo sent; stamp shown on top of it	No extra address lookup or re-saving on the phone
Freeze screen	0.9 seconds	0.25 seconds	Shorter pause before returning to the attendance screen
Final photo	One try, 70% quality	Up to 3 tries, 90% quality	A busy camera no longer ends the scan with "Failed to capture photo"

> "The face box appears sooner, a blink is usually caught on the first try, and the photo is taken the moment it happens - while the GPS reading is already being taken in the background. The web portal's scanner uses the same blink rules and the same fast server path."

7. APIs and Technologies Used

> Important point for the panel: the system uses NO PAID APIs and NO API KEYS. All face recognition runs on our own server - employee photos are never sent to a third-party cloud service.

API / Technology	What it's for	Notes for the panel
Our own REST API (NestJS backend)	All the logic: face and blink detection, face matching, geofence checks, saving attendance and flagged scans	This is the system we built - the phone and admin website are just clients of it

face-api.js (open-source library)	Face detection, eye points (for the blink check), and the 128-number face fingerprint	Runs on OUR SERVER with pre-trained models (TinyFaceDetector, 68-point landmarks, FaceRecognitionNet). The admin website also runs it in the browser to guide HR during face registration. A library we host, not an outside service
node-canvas	Lets face-api.js read images on the server; also used for the brightness check and auto-brighten	Open-source, runs on our server
Expo Camera	Uses the phone's front camera for the blink pictures and the final photo	Built-in device API, free
Expo Image Manipulator	Shrinks each blink picture to 192 pixels before it is sent	Built-in device API, free - one of the main speed improvements
Expo Location	Gets GPS coordinates and accuracy, and turns coordinates into a readable address for the on-screen stamp	Uses the phone's own GPS - no Google Maps API needed
Browser Geolocation API	Gets the location when an employee clocks in from the web portal	Built into the browser; less accurate without GPS, which is why the web allows 2,000 meters of GPS error
OpenStreetMap and Leaflet	The interactive maps where admins draw geofence circles (admin website and supervisor screens)	Free, community-run map data - no API key
OpenStreetMap Nominatim	Turns coordinates into a readable address for the DTR and reports; place search when creating an area	Free, no API key
Carto map tiles (free tier)	The mini-map on the attendance photo stamp	Free, no-key map images; used because OpenStreetMap's own tile server blocks direct mobile-app traffic
Google Maps (link only)	"Open in Google Maps" link when a reviewer checks a scan's location	Just a web link - no Google Maps API or key
AWS Rekognition	NOT USED - only a placeholder hook exists in the code	Honest answer if asked: cloud-grade video liveness was scoped out; the connection point is ready as future work

> "Face recognition is done by face-api.js running on our own server, GPS comes from the phone through Expo Location, and maps come from free OpenStreetMap and Carto tiles - so the entire system runs with zero API costs and employee biometric data never leaves our server."